Relazione Tecnica

Scopo della relazione

Lo scopo della relazione è illustrare il funzionamento del codice prodotto sia in caso di corretto funzionamento, ma soprattutto in caso di errore.

- **Test** : **change_current_directory** (non funzionante) :
  - **Scopo** :
    Verificare che la shell capisca che la cartella in cui sto provando ad entrare non esiste.
  - **Situazione iniziale** :
    Ambiente in cui non è presente la cartella “pippo”.
  - **Linea inviata alla microbash** :
    cd pippo
  - **Risultato atteso :**
    Messaggio di errore ma senza chiudere il programma

- **Test** : **change_current_directory** (funzionante) :
  - **Scopo :**
    Verificare che la shell riesca a tornare indietro di una cartella nell’albero delle directory
  - **Situazione iniziale :**
    Ambiente iniziale o in qualsiasi cartella disponibile a meno che non sia l’ultima dell’albero delle directory.
  - **Linea inviata alla microbash :**
    cd ..
  - **Risultato atteso :**
    Nessun tipo di messaggio stampato sullo standard output e spostamento di una cartella indietro rispetto al path indicato a sinistra.

- **Test** : **change_current_directory** (funzionante) :
  - **Scopo :**
    Verificare che la shell sia in grado di cambiare directory.
  - **Situazione iniziale :**
    Ambiente in cui è presente la cartella pippo.
  - **Linea inviata alla microbash :**
    cd pippo
  - **Risultato atteso :**
    Nessun messaggio di errore ed a sinistra cambia il path della bash.

- **Test : run_child** (non funzionante) :
  - **Scopo :**
    Verificare il comportamento della bash inserendo un comando non valido
  - **Situazione iniziale :**
    Ambiente iniziale della bash.
  - **Linea inviata alla microbash :**
    <qualsiasi comando non esistente>
  - **Risultato atteso :**
    Messaggio di errore con esecuzione del comando inserito fallita.

- **Test : run_child** (funzionante) :
  - **Scopo :**
    Verificare il comportamento della bash inserendo un comando valido
  - **Situazione iniziale :**
    Ambiente iniziale della bash.
  - **Linea inviata alla microbash :**
    <qualsiasi comando esistente>
  - **Risultato atteso :**
    Nessun messaggio di errore ed esecuzione del comando inserito.

- **Test : redirect** (funzionante) :
  - **Scopo :**
    Verificare che la funzione effettui la Dup sul fd.
  - **Situazione iniziale :**
    Esecuzione di un comando con I/O redirection.
  - **Linea inviata alla microbash :**
    ls -l >file.txt
  - **Risultato atteso:**
    Nessun errore e ridirezione output(in questo caso) su file.txt.

- **Test : redirect** (non funzionante) :
  - **Scopo :**
    Verificare che la funzione ritorni un errore in caso di fallimento della Dup.
  - **Situazione iniziale :**
    Invio di un comando con redirezione dell I/O.
  - **Linea inviata alla microbash :**
    ...
  - **Risultato atteso:**
    Messaggio di errore con conseguente terminazione del programma.

- **Test : getcwd** (funzionante) :
  - **Scopo :**
    Verificare che la funzione ritorni la cartella di lavoro corrente.
  - **Situazione iniziale :**
    Lancio dell'eseguibile che deve provvedere a stampare la current working directory.
  - **Linea inviata alla microbash :**
  - **Risultato atteso :**
    Una stringa a sinistra del terminale del tipo :
    /home/mammo/SETI/microbash-students $

- **Test :** **getcwd** (non funzionante) :
  - **Scopo :**
    Verificare che la funzione torni NULL se si accede ad una cartella che non permette la lettura.
  - **Situazione iniziale :**
    Lancio dell'eseguibile in una cartella senza il permesso di lettura.
  - **Linea inviata alla microbash :**
  - **Risultato atteso :**
    La stringa: *ERROR IN THE getcwd* e chiusura del programma

- **Test** : **free_command** (funzionante) :
  - **Scopo :**
    Verificare che la funzione liberi in modo corretto la struttura contenente il comando.
  - **Situazione iniziale :**
    Esecuzione di un comando con la sua conseguente *free_command.*
  - **Linea inviata alla microbash :**
    ls
  - **Risultato atteso :**
    Una volta terminato il programma con ctrl-d non avere memory-leak.

- **Test** : **free_command** (non funzionante) :
  - **Scopo :**
    Verificare che la funzione non liberi in modo corretto la struttura contenente il comando.
  - **Situazione iniziale :**
    Esecuzione di un comando con la sua conseguente *free_command.*
  - **Linea inviata alla microbash :**
    ls
  - **Risultato atteso :**
    Una volta terminato il programma con ctrl-d si presenteranno vari memory-leak.

- **Test** : **free_line** (funzionante) :
  - **Scopo :**
    Verificare che la funzione liberi in modo corretto la struttura contenente i comandi(la linea).
  - **Situazione iniziale :**
    Esecuzione di una linea di comando con la sua conseguente *free_line.*
  - **Linea inviata alla microbash :**
    ls | grep M
  - **Risultato atteso :**
    Una volta terminato il programma con ctrl-d non avere memory-leak.

- **Test : free_line** (non funzionante) :
  - **Scopo :**
    Verificare che la funzione non liberi in modo corretto la struttura contenente i comandi(la linea).
  - **Situazione iniziale :**
    Esecuzione di una linea di comando con la sua conseguente *free_line.*
  - **Linea inviata alla microbash :**
    ls | grep M
  - **Risultato atteso :**
    Una volta terminato il programma con ctrl-d si presenteranno vari memory-leak.

- **Test : execute_line** (funzionante) :
  - **Scopo :**
    Esecuzione di tutti i comandi presenti nella linea.
  - **Situazione iniziale :**
    invio di una linea di comandi alla bash.
  - **Linea inviata alla microbash :**
    cat <nome_file> | grep <parola> ><altro_file>
  - **Risultato atteso :**
    Riuscita del comando con il risultato presente in <altro_file>.
- **Test : execute_line** (non funzionante) :
  - **Scopo :**
    Esecuzione di tutti i comandi presenti nella linea e gestione errori.
  - **Situazione iniziale :**
    invio di una linea di comandi alla bash che provano a effettuare operazioni su un file senza permessi.
  - **Linea inviata alla microbash :**
    cat <nome_file> | grep <parola> ><altro_file>
  - **Risultato atteso :**
    Non riuscita del comando con i messaggi di errore opportuni una volta terminati i figli.
- **Test : check_cd** (funzionante) :
  - **Scopo :**
    Verificare che la funzione ritorni CHECK_OK se il comando cd è scritto correttamente.
  - **Situazione iniziale :**
    Esecuzione del comando cd sintatticamente corretto.
  - **Linea inviata alla microbash :**
    cd .. oppure cd <nome_cartella>
  - **Risultato atteso :**
    Cambio della cwd da quella corrente a quella contenente la vecchia cartella corrente.

- **Test** : **check_cd** (non funzionante) :
  - **Scopo :**
    Verificare che la funzione ritorni CHECK_FAILED se il comando cd non è scritto correttamente.
  - **Situazione iniziale :**
    Esecuzione di una linea di comando cd sintatticamente sbagliato.
  - **Linea inviata alla microbash :**
    ls | cd .. oppure cd foo bar oppure cd .. | ls
  - **Risultato atteso :**
    Stampa dell'errore commesso e la conseguente non esecuzione della linea di comando o del comando.

```
/home/mammo/SETI/microbash-students $ cd .. foo
ERR:: Expect max 1 argument for CD command
/home/mammo/SETI/microbash-students $ ls | cd
ERR:: found CD command in wrong position on cmd-line
/home/mammo/SETI/microbash-students $ cd >pippo.txt
ERR:: Expect max 1 argument for CD command
/home/mammo/SETI/microbash-students $ ls | grep M | cd ..
ERR:: found CD command in wrong position on cmd-line
/home/mammo/SETI/microbash-students $
```

- **Test** : **variabili d'ambiente** (funzionante) :
  - **Scopo :**
    Verificare la corretta espansione delle variabili d'ambiente
  - **Situazione iniziale :**
    Esecuzione del comando *echo $<nome_variabile>*.
  - **Linea inviata alla microbash :**
    echo $home
  - **Risultato atteso :**
    Stampa dell'espansione della variabile home.

- **Test : variabili d'ambiente** (non funzionante) :
  - **Scopo :**
    Verificare che l'espansione di una variabile d'ambiente non esistente sia " ".
  - **Situazione iniziale :**
    Esecuzione del comando echo $<nome_variabile> dove nome_variabile non ha corrispondenza.
  - **Linea inviata alla microbash :**
    echo ciao $pippo mondo
  - **Risultato atteso :**
    Stampa *ciao mondo* (con 2 spazi tra le parole).
- **Test : variabili d'ambiente** (non funzionante) :
  - **Scopo :**
    Verificare che l'espansione di una variabile d'ambiente non esistente sia "".
  - **Situazione iniziale :**
    Esecuzione del comando *echo $<nome_variabile>* dove nome_variabile non ha corrispondenza.
  - **Linea inviata alla microbash :**
    echo ciao $pippo mondo
  - **Risultato atteso :**
    Stampa *ciao mondo* (con 2 spazi tra le parole).
- **Test** : **check_redirections** (funzionante) :
  - **Scopo :**
    Verificare che la funzione ritorni CHECK_OK se il comando redirezione (< o >) è scritto correttamente.
  - **Situazione iniziale :**
    Esecuzione del comando ridirezione sintatticamente corretto.
  - **Linea inviata alla microbash :**
    cat <nome_file> | grep <parola> >bar
    cat </proc/cpuinfo | grep processor
    grep e <microbash.c >bar
  - **Risultato atteso :**
    Selezionare dei valori presi da un file con il simbolo '<' .

Oppure stampare dei valori in un file con il simbolo '>'.

- **Test** : **check_redirections** (non funzionante) :
  - **Scopo :**
    Verificare che la funzione ritorni CHECK_FAILED se il comando redirezione non è scritto correttamente o è nella posizione sbagliata.
  - **Situazione iniziale :**
    Esecuzione di una linea di comando sintatticamente sbagliato.
  - **Linea inviata alla microbash :**
    ```
    ><nomefile>
    cat <nomefile> | grep e <bar | ls -l
    grep micro > bar (spazio prima di bar)
    ```
  - **Risultato atteso :**
    Stampa dell'errore commesso e la conseguente non esecuzione della linea di comando o del comando.

- **Test : wait_for_children** (funzionante) :
  - **Scopo :**
    aspettare tutti i figli creati
  - **Situazione iniziale :**
    invio di una linea di comandi alla bash
  - **Linea inviata alla microbash :**
    cd pippo
    cat </proc/cpuinfo | grep processor | wc -l
  - **Risultato atteso :**
    esecuzione dei comandi senza alcun ritorno di errore dalla funzione.

- **Test : wait_for_children** (non funzionante) :
  - **Scopo :**
    aspettare tutti i figli creati
  - **Situazione iniziale :**
    invio di una linea di comandi errati alla bash oppure un comando corretto che viene ucciso
  - **Linea inviata alla microbash :**
    pippo pippo
    cat
  - **Risultato atteso :**
    Nel primo caso ritorno di un exit status perché un figlio è stato terminato.
    Nel secondo caso ritorno un segnale di terminazione da un altro processo.